

# When to Clean the Machine: A Rust Model of Codebase Rationalization in AI-Native Startups

Research Proposal

Briac Sockalingum

May 27, 2026

## Abstract

The proliferation of AI-assisted and “vibe-coded” software development has created a new class of dynamic replacement problem for firms: when to rationalize fragmented, AI-generated codebases into cleaner, human-audited architectures. This paper adapts Rust (1987)’s bus engine replacement model to the software context, treating codebase rationalization as a lumpy, partially irreversible investment decision made under uncertainty about future maintenance costs and failure risk. I propose a hybrid data strategy combining scraped signals from public GitHub repositories—commit authorship, code churn, complexity metrics, refactor branch events—with ground-truth labeling from a panel of cooperating startups to validate the measurement framework. The state space is collapsed to a scalar *Codebase Quality Index* (CQI), constructed via a first-stage factor model, making structural estimation tractable via Nested Fixed Point (NFXP) maximum likelihood or Hotz-Miller Conditional Choice Probability (CCP) methods. The structural goal is to recover the implied cost of operating degraded, AI-generated code versus the fixed cost of deliberate rationalization, and to characterize the replacement policy as a function of observable codebase state. The paper contributes (i) the first DDC model of codebase remediation and technical debt rationalization as a lumpy replacement investment, extending Cho (2011)’s hardware replacement framework to AI-generated software; (ii) a reusable measurement framework for inferring AI-versus-human coding posture from repository data; and (iii) a structural foundation for evaluating policy counterfactuals such as compliance mandates or internal governance standards around AI-generated code.

# Contents

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Motivation</b>                                      | <b>3</b>  |
| <b>2</b> | <b>Related Literature</b>                              | <b>4</b>  |
| 2.1      | Technical Debt as an Economic Problem . . . . .        | 4         |
| 2.2      | Structural Dynamic Discrete Choice . . . . .           | 5         |
| 2.3      | IT Replacement and Technology Adoption . . . . .       | 6         |
| 2.4      | Economics of AI-Assisted Development . . . . .         | 6         |
| 2.5      | AI Code Measurement and Governance . . . . .           | 7         |
| <b>3</b> | <b>Research Questions</b>                              | <b>8</b>  |
| <b>4</b> | <b>Conceptual Framework</b>                            | <b>8</b>  |
| 4.1      | The Rust Model Applied to Codebase Decisions . . . . . | 8         |
| 4.2      | The Scalar CQI as State Variable . . . . .             | 9         |
| 4.3      | The Conditional Independence Assumption . . . . .      | 10        |
| <b>5</b> | <b>Empirical Strategy and Data</b>                     | <b>10</b> |
| 5.1      | Sample Construction . . . . .                          | 10        |
| 5.2      | Constructing the Codebase Quality Index . . . . .      | 11        |
| 5.3      | Identifying the Replacement Event . . . . .            | 11        |
| 5.4      | Identification . . . . .                               | 12        |
| <b>6</b> | <b>Contributions</b>                                   | <b>13</b> |
| 6.1      | Substantive . . . . .                                  | 13        |
| 6.2      | Methodological . . . . .                               | 13        |
| 6.3      | Policy Relevance . . . . .                             | 13        |
| <b>7</b> | <b>Feasibility and Timeline</b>                        | <b>14</b> |

# 1 Motivation

Software development has undergone what economists would recognize as a general purpose technology (GPT) transition. The deployment of AI coding assistants—GitHub Copilot, Cursor, Claude Code—has made functional software increasingly accessible to non-specialists, producing a pattern known colloquially as “vibe coding”: iterative, prompt-driven code generation without architectural planning. By 2024, AI tools had penetrated 90% of Fortune 100 technology teams (Microsoft Corporation, 2025), and field experiments find real-world productivity gains of 14–26% in completed pull requests (Cui et al., 2026).

These short-run velocity gains generate a structural liability. AI-generated code accumulates technical debt rapidly: it exhibits high cyclomatic complexity, weak test coverage, opaque dependency chains, and a tendency toward fragmentation across modules that were never designed to cohere (Li et al., 2015; Liu et al., 2026). Crucially, the accumulation is not linear. Selvanayagam et al. (2026) document a latency-then-acceleration pattern: LLM-integration repositories remain technically debt-free 2.4 times longer than comparable non-LLM repositories (median 492 vs. 204 days before debt onset), but once debt begins, accumulation accelerates. Avgeriou et al. (2024) confirm a “broken windows” mechanism—existing technical debt density significantly increases the propensity to introduce new debt—consistent with a convex degradation function.

The resulting maintenance cost curve is initially low but accelerating as codebase age and AI-generation share grow, until the firm faces a discrete, lumpy choice: continue operating degraded infrastructure at rising cost, or invest in a deliberate rationalization sprint that resets the state at significant short-run cost. Borg et al. (2024) document that returns to codebase maintainability are convex at the high end—crossing a quality threshold yields disproportionate downstream productivity gains—implying that rationalization has non-linear payoffs, not merely avoided future costs. This dynamic is precisely the structure of Rust (1987)’s bus engine replacement model.

Yet no empirical work has applied structural dynamic choice methods to software engineering decisions. This paper does so, exploiting the emergence of vibe coding as both the source of the problem and—through the rich digital traces it leaves in version control systems—the source of the data.

The paper also speaks to the broader IT productivity literature. Brynjolfsson and Hitt (2003) establish that IT investment yields productivity contributions up to five times larger in the long run than in the short run, mediated by complementary organizational investments.

Brynjolfsson et al. (2021) formalize this as the “productivity J-curve”: general purpose technologies require complementary intangible investments before productivity gains materialize. Codebase rationalization is precisely this complementary investment: AI-generated code requires rationalization before full productivity returns to AI tooling are realized.

## 2 Related Literature

This paper sits at the intersection of five bodies of work that have until now developed independently.

### 2.1 Technical Debt as an Economic Problem

The concept of technical debt originates with Cunningham (1992), who introduced the debt metaphor to describe the tradeoff between shipping quickly and shipping well. Early quantitative evidence on maintenance costs comes from Lientz and Swanson (1980), who found that 75–80% of programmer effort was consumed by maintenance rather than new development. Banker et al. (1993) provided the first field estimate of the complexity-maintenance cost link, finding that more complex software carried approximately a 25% cost disadvantage. Banker et al. (1998) extended this to show that poor design choices at development time are a robust predictor of long-run maintenance costs.

Li et al. (2015), in a systematic mapping of 94 peer-reviewed studies, identify ten types of technical debt, with code and architectural debt most directly linked to long-run system reliability. The richest econometric evidence is Ramasubbu and Kemerer (2016), whose competing-risks analysis of a 10-year longitudinal dataset covering a commercial enterprise system across 48 client firms finds that modular maintenance was approximately 53% more effective than architectural maintenance in reducing client-error failure probability but produced an 83% increase in vendor-error failures. Banker et al. (2021a) find an inverted-U relationship between technical debt accumulated during accounting information system deployment and firm-level performance.

For dynamic modeling, a critically important empirical finding is the broken windows mechanism: Avgeriou et al. (2024), in a pre-registered controlled experiment with 29 developers, find that higher existing technical debt density significantly increases the propensity to introduce new technical debt through three behavioral channels—re-implementation rather than reuse, selection of non-desirable designs, and mirroring of existing poor patterns. This implies a positive feedback loop and motivates a convexity term in the state transition function (see Section 4).

## 2.2 Structural Dynamic Discrete Choice

Rust (1987) is the canonical application: using 10 years of monthly data on 104 buses, he recovers structural cost parameters of a bus engine replacement decision. The key methodological innovations are (i) the Nested Fixed Point (NFXP) algorithm for solving the dynamic programming problem at each likelihood evaluation; (ii) the conditional independence (CI) assumption, which makes the state transition Markovian; and (iii) i.i.d. Type-I Extreme Value errors, enabling closed-form logistic choice probabilities. Rust (1994) establishes the framework’s limits: NFXP is practical only for state spaces up to approximately 10,000 states.

Magnac and Thesmar (2002) establish a fundamental non-identification result: the discount factor  $\beta$ , shock distribution  $G(\cdot)$ , and reference utility cannot be jointly identified without normalization. The common practice of calibrating  $\beta$  (e.g.,  $\beta = 0.9999$  in Rust) is not merely computational convenience but a necessary identification normalization.

Hotz and Miller (1993) transform DDC estimation via the CCP inversion theorem: given nonparametrically estimated choice probabilities  $\hat{P}(x)$ , one can recover continuation value differences without solving the full dynamic program. Hotz et al. (1994) extend this to settings without absorbing terminal states. Arcidiacono and Miller (2011) introduce the finite-dependence result, showing that value function differences become negligible after  $K$  periods, substantially reducing first-stage requirements, and handle unobserved heterogeneity via an EM algorithm. Aguirregabiria and Mira (2002) propose the NPL (Nested Pseudo-Likelihood) estimator, which interleaves pseudo-likelihood maximization with policy iteration.

For high-dimensional state spaces, Keane and Wolpin (1994) develop a simulation-interpolation (SMCP) approach that approximates the value function from a random subsample of state points, enabling tractable backward induction even when the state space is theoretically vast. Barzegary and Yoganarasimhan (2022) propose the RePaD algorithm, which maps high-dimensional nuisance state variables to a low-dimensional partition via recursive splitting, maintaining linear time complexity in state-space dimension. Nguyen (2025) develop a neural network estimator that replaces the value function fixed-point solve with a neural approximation, achieving  $\sqrt{n}$ -consistency under a Neyman orthogonality condition. Arcidiacono and Ellickson (2011) synthesize practical implementation advice: prefer CCP approaches when computationally constrained, and design the state space to start low-dimensional and add variables incrementally.

## 2.3 IT Replacement and Technology Adoption

The closest methodological predecessor is Cho (2011), who applies the Rust framework to mainframe computer investment at a telecommunications firm (Korea, 1989–1999), covering 105 mainframe systems over 10 years. Cho’s critical innovation is a parametric NLS-NFXP approach that makes a 4-dimensional continuous state space computationally feasible without grid discretization. The paper decomposes a 30-fold capacity increase over the decade into Moore’s Law cost decline (17–21%) and demand growth (11–13%). Cho (2011) is the closest existing methodological antecedent to the present paper; its key architectural choices—parametric approximation for continuous states, economic depreciation as the degradation driver—are directly transferable with AI-debt accumulation replacing Moore’s Law.

Han and Xu (2019) apply a Rust-style model to the adoption decision of Python 3 compatibility by 3,397 PyPI packages. Their key structural finding: the cost of adopting Python 3 attributable to incompatible upstream dependencies is approximately three times the cost of updating one’s own code—a direct analog to the codebase rationalization setting, where incompatible dependencies map to the cost of cleaning third-party AI-generated modules. Notably, their estimated mean annual discount factor of approximately 0.486 is far more myopic than the canonical 0.999, suggesting software maintainers may be substantially more short-sighted than Harold Zurcher.

Ryan and Tucker (2012) model videocalling adoption within a single multinational bank, using ex-post calling behavior to identify network benefits separately from adoption cost correlations. Their identification innovation—using revealed post-adoption behavior to separately identify cost components—is applicable to the codebase setting: ex-post maintenance logs (incident rates, time-to-fix) could identify the cost of operating degraded code independently of the rationalization decision.

## 2.4 Economics of AI-Assisted Development

The most credible productivity evidence comes from pre-registered field experiments. Peng et al. (2023) find a 55.8% reduction in task completion time for GitHub Copilot access in a laboratory RCT ( $N = 95$  developers). Cui et al. (2026), in three pre-registered field RCTs at Microsoft, Accenture, and an anonymous Fortune 100 manufacturer ( $N = 4,867$  developers), find 26.1% more completed pull requests per week, with strong heterogeneity: less experienced developers gain 27–39%, senior developers 8–16%. Brynjolfsson et al. (2025) document 14–15% productivity gains from AI customer service agents, with larger gains for novices.

On code quality, the evidence is consistently negative. Pearce et al. (2022) find that approximately 40% of AI-generated programs across 89 security-relevant scenarios are CWE-vulnerable. Perry et al. (2023) show that developers with AI assistant access wrote significantly less secure code than controls and were more confident in their code’s security—a false confidence effect that reduces the likelihood of seeking code review precisely when it is most needed. Liu et al. (2026) analyze 304,362 verified AI-authored commits from 6,275 repositories and find that 24.2% of AI-introduced issues persist to the latest revision, becoming permanent technical debt.

The synthesis insight is a divergence between productivity and quality: real-world productivity gains of 14–26% materialize within weeks, while technical debt accumulates with a latency and then accelerates (Selvanayagam et al., 2026). This pattern maps directly to the Rust framework: a period of apparently cheap operation (low mileage), followed by accelerating maintenance costs, eventually triggering a discrete rationalization decision.

## 2.5 AI Code Measurement and Governance

Ghaleb (2026) achieve 97.2% macro F1 for AI agent fingerprinting across five agents (OpenAI Codex, GitHub Copilot, Devin, Cursor, Claude Code) using 41 features from pull request metadata. However, two structural limitations matter critically for this paper: (1) the model cannot detect inline completion use—the dominant mode of AI-assisted development—and (2) Claude Code achieves only 57% recall (the lowest of the five agents). The true false-negative rate for AI authorship detection in production repositories is therefore structurally very high, likely exceeding 80% for inline completions.

This measurement limitation implies that the AI-generation share state variable will be systematically biased toward zero for firms that do not voluntarily disclose AI assistance, creating downward bias in estimated cost parameters. The validation stage with cooperating startups (see Section 5) is designed to calibrate the signal composite and bound measurement error.

On governance, the EU AI Act (enacted August 2024) has a phased compliance timeline: prohibited practices applied from February 2025, with full high-risk provisions from August 2027 (European Parliament and Council of the European Union, 2024). AI coding tools are not classified as high-risk systems under Annex III. Importantly for instrument design, the Act’s rollout was publicly telegraphed from the April 2021 Commission draft through 2023 trilogue negotiations, creating severe anticipation bias for any instrument that treats its announcements as shocks (see Section 5.4).

### 3 Research Questions

1. **Replacement policy.** How do AI-native startups dynamically decide when to rationalize a vibe-coded codebase? What codebase state variables—age, complexity, incident history—drive the replacement threshold?
2. **Structural costs.** What is the implied per-period cost of operating degraded, AI-generated code relative to rationalized code, and how large is the fixed cost of executing a rationalization sprint? What do these structural parameters imply about the social and private returns to cleaning AI-generated codebases?
3. **Policy counterfactuals.** How would the replacement policy and social welfare change under alternative regulatory or governance environments—for example, mandatory code quality standards for AI-assisted development, SOC 2 audit requirements triggered by enterprise contracts, or tax treatment of rationalization sprints as capital investment?

## 4 Conceptual Framework

### 4.1 The Rust Model Applied to Codebase Decisions

I adapt the Rust (1987) model as follows. In each period  $t$ , the firm observes the scalar Codebase Quality Index (CQI)  $q_t \in \mathcal{Q} \subset \mathbb{R}$  (constructed as described in Section 5) and chooses an action  $a_t \in \{0, 1\}$ , where  $a_t = 0$  denotes continuation (operate the current codebase) and  $a_t = 1$  denotes replacement (execute a rationalization sprint). The per-period utility is:

$$u(q_t, a_t, \theta) = \begin{cases} -c(q_t, \theta_c) + \varepsilon_t(0) & \text{if } a_t = 0 \\ -[\kappa + \varepsilon_t(1)] & \text{if } a_t = 1 \end{cases} \quad (1)$$

where  $c(q_t, \theta_c)$  is the flow maintenance cost of operating a codebase in state  $q_t$ , parameterized by  $\theta_c$ ;  $\kappa$  is the fixed cost of rationalization (developer time, productivity loss during refactor, risk of introducing new bugs); and  $\varepsilon_t = (\varepsilon_t(0), \varepsilon_t(1))$  are i.i.d. Type-I Extreme Value shocks that generate logistic choice probabilities (Rust, 1987). Following empirical evidence on convex degradation dynamics (Borg et al., 2024; Avgeriou et al., 2024), I parameterize  $c(q_t, \theta_c)$  with a quadratic or exponential functional form rather than a linear approximation.

The state transition follows:

$$q_{t+1} = \begin{cases} q_t - \delta(q_t, \eta_t) & \text{if } a_t = 0 \quad (\text{codebase degrades}) \\ q_0 + \eta_t & \text{if } a_t = 1 \quad (\text{partial reset}) \end{cases} \quad (2)$$

where  $\delta(q_t, \eta_t) \geq 0$  is a degradation function—convex in  $q_t$  to capture the broken windows mechanism (Avgeriou et al., 2024)—and  $\eta_t$  is a stochastic innovation to the state. I allow  $q_0$  to be partial, not complete, rationalization: a firm might remediate 60–80% of AI-generated components, leaving residual debt; the reset is to  $q^* + \eta_t$  where  $q^* < \bar{q}$  in most practical cases.

The firm’s value function satisfies the Bellman equation:

$$V(q_t) = \max_{a_t \in \{0,1\}} \{u(q_t, a_t, \theta) + \beta \mathbb{E}[V(q_{t+1}) \mid q_t, a_t]\} \quad (3)$$

Structural parameters  $\theta = (\theta_c, \kappa, \beta)$  are recovered via NFXP maximum likelihood (Rust, 1987) on the scalar state  $q_t$ , or via the Hotz-Miller CCP estimator (Hotz and Miller, 1993; Arcidiacono and Miller, 2011), which avoids solving the full dynamic program at each likelihood evaluation and is substantially more tractable when replacement events are sparse. The discount factor  $\beta$  is calibrated rather than estimated, following the Magnac-Thesmar non-identification result (Magnac and Thesmar, 2002); I report sensitivity analysis over a range  $\beta \in [0.9, 0.999]$  following the myopic discount factor estimated by Han and Xu (2019) for software maintainers.

## 4.2 The Scalar CQI as State Variable

A five-dimensional state vector—cyclomatic complexity, AI-generation share, test coverage, incident history, dependency age—would render both NFXP and Hotz-Miller CCP infeasible at any realistic sample size. With five continuous dimensions discretized to five bins each, the joint state space has  $5^5 = 3,125$  cells; with 300–1,800 total observations, most cells contain zero observations. This is not a minor estimation challenge but a structural impossibility (Rust, 1994; Aguirregabiria and Mira, 2010).

The fix is to aggregate to a scalar sufficient statistic. I construct the CQI  $q_t$  as a weighted composite of the five raw signals, estimated in a first-stage measurement model (factor analysis or index number approach), analogously to Keane and Wolpin (1994)’s SMCP approach for human capital models with numerous raw state signals and Cho (2011)’s scalar capacity state. The dynamic model then has a scalar state, making standard NFXP feasible

and CCP estimation tractable. The SATD latency pattern (Selvanayagam et al., 2026) and the Volume-Quality Inverse Law (Zhu et al., 2026) together suggest that the CQI state transition has a convex degradation function—the key structural feature needed to make the replacement decision non-trivial and well-identified.

### 4.3 The Conditional Independence Assumption

Rust’s CI assumption requires that  $s_{t+1} \perp (s_0, \dots, s_{t-1}) \mid (s_t, a_t)$ : the state transition depends only on the current state and action, not the full history. In the bus engine context this is plausible; in the codebase context it is substantially less so. Prior failed rationalization attempts, team turnover, and historical tooling choices plausibly create history-dependence in how states evolve.

I address this in three ways: (i) the scalar CQI aggregates history into the current state, making the Markov assumption more defensible than for any individual raw signal; (ii) firm fixed effects in the measurement panel capture stable organizational features that generate history-dependence; and (iii) I follow Arcidiacono and Miller (2011)’s finite-dependence approach, which provides a robustness check by allowing value function differences to depend on a limited history. I report the Rust (1987) LM test for the CI assumption on the scalar state and, if rejected, adopt the finite-dependence correction.

## 5 Empirical Strategy and Data

### 5.1 Sample Construction

The sample consists of early-stage startups (seed to Series A) that (i) openly use AI coding tools, (ii) maintain public GitHub repositories, and (iii) can be linked to observable firm-level characteristics via Crunchbase or LinkedIn. I target a panel of  $N \in [50, 150]$  firms observed quarterly over 6–12 quarters (2023 Q1 to 2025 Q4), yielding between 300 and 1,800 firm-quarter observations.

Sample construction proceeds in two stages:

1. **Scrapping stage.** Identify candidate firms via GitHub search (repositories with high AI-tool adoption signals), Crunchbase funding filters (seed/Series A, software, founded 2021–2024), and public statements about AI-assisted development. Collect quarterly repository snapshots via the GitHub REST API.
2. **Validation stage.** Contact a subset of 10–20 firms for voluntary participation in a short

survey or dashboard-sharing agreement to ground-truth the scraped labels—particularly the replacement event classification and the AI-versus-human authorship share. This hybrid design is faster than pure cold outreach and more defensible than pure scraping. The validation sample informs the measurement model used to construct the CQI for the full scraped panel.

## 5.2 Constructing the Codebase Quality Index

The CQI  $q_t$  is constructed as a weighted composite of five raw signals, estimated in a first-stage factor model:

- **Complexity index:** cyclomatic complexity per module (McCabe, 1976; ISO/IEC 25010), computed via SonarQube or Radon from repository history.
- **AI-generation share:** fraction of commits bearing AI authorship signals (Ghaleb, 2026), measured as a rolling quarterly average; calibrated against the validation panel to bound false-negative bias from undisclosed inline completions.
- **Relative code churn:** churn normalized by component size (Nagappan and Ball, 2005), discriminating fault-prone components at 89% accuracy.
- **Incident rate:** cumulative count of production outages, security bug reports, and dependency vulnerability flags in the trailing four quarters.
- **Dependency vulnerability count:** count of third-party dependencies with active CVEs, a direct proxy for security risk from outdated packages.

The factor model’s first principal component aggregates these into a scalar CQI. Weights are estimated on the validation panel where ground-truth is available, then applied to the full scraped panel with bootstrapped standard errors to propagate measurement uncertainty into the structural estimates.

## 5.3 Identifying the Replacement Event

The replacement event ( $a_t = 1$ ) is identified as a deliberate rationalization sprint: a discrete, bounded period during which the firm substantially rewrites or restructures AI-generated components. In repository data, this is signaled by:

- A dedicated branch with name patterns `refactor/`, `cleanup/`, `rewrite/`, or `rationalize/`.
- A sharp, sustained drop in AI-authorship share within a quarter, accompanied by a spike in human-authored commits.

- Bulk deletion of AI-generated modules followed by new human-authored replacements (high negative churn followed by positive churn with different authorship signatures).
- Explicit pull request descriptions referencing technical debt remediation, code quality gates, or compliance with new internal standards.

Whether the distribution of rationalization events is bimodal (supporting the lumpy assumption) or unimodal (supporting continuous improvement) is an empirical question I address in the descriptive stage before structural estimation.

## 5.4 Identification

The structural parameters are identified from within-firm variation in the replacement decision across periods with varying CQI, under the CI assumption (Section 4).

For robustness, I exploit one primary source of quasi-exogenous variation:

1. **SOC 2 compliance shocks.** SOC 2 Type II certification requires change management controls (CC8), code review procedures, and separation of duties for production deployments. Critically, SOC 2 adoption events are typically triggered by firm-specific enterprise customer contract requirements—not by a publicly anticipated regulatory calendar—making them plausibly exogenous to the current codebase state. Firms under SOC 2 audit pressure face a higher cost of continuing to operate unaudited AI-generated code, raising the value of rationalization. I exploit cross-firm variation in the timing of SOC 2 certification milestones, instrumented by contract-driven adoption events identified from Crunchbase and LinkedIn announcements.

I explicitly rule out the EU AI Act as a primary instrument. While the Act’s phased provisions (February 2025 for prohibited practices, August 2027 for high-risk systems) create compliance pressure, the Act was publicly telegraphed from April 2021 through 2023 trilogue negotiations (European Parliament and Council of the European Union, 2024), creating severe anticipation bias. Forward-looking firms could have been rationalizing code in anticipation of compliance for years before any formal deadline. Additionally, AI coding tools are not classified as high-risk under Annex III, meaning the exclusion restriction is theoretically contested. I use cross-country variation in AI regulation (EU-incorporated versus non-EU startups in the same cohort) only as an auxiliary robustness check, modeled on Bertomeu et al. (2025)’s use of Italy’s ChatGPT ban as a natural experiment.

## 6 Contributions

### 6.1 Substantive

This paper provides the first structural dynamic discrete choice model of codebase remediation and technical debt rationalization as a lumpy replacement investment, extending Cho (2011)’s hardware replacement framework to the novel setting of AI-generated software architecture. It is distinct from Cho (2011) (hardware, not software; Moore’s Law, not AI-debt accumulation), Han and Xu (2019) (package adoption, not architecture rationalization), and Banker et al. (2021b) (reduced-form technical debt correlates, not structural dynamic choice).

The structural estimates quantify, in dollar-equivalent terms, the cost of operating vibe-coded infrastructure and the private return to rationalization—parameters with direct relevance to founders, engineering leads, and investors who currently make these decisions under pure rule-of-thumb. They also speak to the J-curve hypothesis: if the AI coding J-curve requires complementary rationalization investments, the structural model identifies the threshold state at which firms optimally undertake them (Brynjolfsson et al., 2021; McElheran et al., 2025).

### 6.2 Methodological

The paper introduces a validated, reusable Codebase Quality Index constructed from repository signals. This measurement contribution stands independently of the structural model: researchers studying AI adoption, code quality, or software productivity can use the CQI as a panel outcome variable without adopting the full DDC framework. The paper also extends Rust-style DDC methods to a setting where “physical wear” is replaced by a digitally-measurable degradation index, opening a methodological template for applying structural IO to security investment, infrastructure migration, and API deprecation.

### 6.3 Policy Relevance

The structural estimates support counterfactual policy analysis: how would replacement rates change under mandatory AI code auditing requirements, standardized technical debt disclosures for venture-backed firms, or tax treatment of rationalization sprints as capital investment? These are live questions in EU AI Act implementation and in emerging enterprise AI governance frameworks (European Parliament and Council of the European Union, 2024).

## 7 Feasibility and Timeline

### 0–3 months

Build the scraping pipeline (GitHub API, Crunchbase), construct the initial sample frame of 50–150 candidate firms, and develop the AI authorship classifier following Ghaleb (2026). Produce descriptive statistics and a first pass at replacement event detection.

### 3–6 months

Run the validation survey with 10–20 firms to calibrate the CQI measurement model and bound false-negative bias. Estimate the first-stage factor model. Produce reduced-form evidence on CQI dynamics and rationalization event distributions before structural estimation.

### 6–12 months

Estimate the Hotz-Miller CCP or NFXP model on the scalar CQI. Incorporate SOC 2 compliance shock instruments. Run counterfactual policy simulations. Produce a working paper draft.

### 12–24 months

Submit to a field journal (*RAND Journal of Economics*, *Journal of Industrial Economics*, *Management Science*, or *Review of Economics and Statistics*) and present at structural IO and technology economics conferences.

## References

- Aguirregabiria, V. and Mira, P. (2002). Swapping the nested fixed point algorithm: A class of estimators for discrete markov decision models. *Econometrica*, 70(4):1519–1543.
- Aguirregabiria, V. and Mira, P. (2010). Dynamic discrete choice structural models: A survey. *Journal of Econometrics*, 156(1):38–67.
- Arcidiacono, P. and Ellickson, P. B. (2011). Practical methods for estimation of dynamic discrete choice models. *Annual Review of Economics*, 3:363–394.
- Arcidiacono, P. and Miller, R. A. (2011). Conditional choice probability estimation of dynamic discrete choice models with unobserved heterogeneity. *Econometrica*, 79(6):1823–1867.
- Avgeriou, P., Kruchten, P., Nord, R. L., Ozkaya, I., and Verdecchia, R. (2024). Managing technical debt: The road ahead. *Empirical Software Engineering*.

- Banker, R. D., Datar, S. M., Kemerer, C. F., and Zweig, D. (1993). Software complexity and maintenance costs. *Communications of the ACM*, 36(11):81–94.
- Banker, R. D., Davis, G. B., and Slaughter, S. A. (1998). Software development practices, software complexity, and software maintenance performance: A field study. *Management Science*, 44(4):433–450.
- Banker, R. D., Liang, Y., and Ramasubbu, N. (2021a). Technical debt and firm performance. *Management Science*, 67(5):3174–3194.
- Banker, R. D., Liang, Y., and Ramasubbu, N. (2021b). Technical debt and firm performance. *Management Science*, 67(5):3174–3194. Note: This paper is correctly attributed to Banker, Liang & Ramasubbu; the label ‘gao2021’ is retained for backward compatibility with reviewer notes.
- Barzegary, E. and Yoganarasimhan, H. (2022). Data-driven state space reduction for estimation of dynamic models. *arXiv preprint arXiv:2208.01476*.
- Bertomeu, J., Lin, S., Liu, J., and Ni, C. (2025). What are the costs of delayed technology adoption? evidence from Italy’s ban of ChatGPT. SSRN Working Paper 4452670.
- Borg, M., Pruvost, I., Mones, E., and Tornhill, A. (2024). Increasing, not diminishing: Investigating the returns of highly maintainable code. *arXiv preprint arXiv:2401.13407*.
- Brynjolfsson, E. and Hitt, L. M. (2003). Computing productivity: Firm-level evidence. *Review of Economics and Statistics*, 85(4):793–808.
- Brynjolfsson, E., Li, D., and Raymond, L. R. (2025). Generative AI at work. *Quarterly Journal of Economics*, 140(2).
- Brynjolfsson, E., Rock, D., and Syverson, C. (2021). The productivity J-curve: How intangibles complement general purpose technologies. *American Economic Journal: Macroeconomics*, 13(1):333–372.
- Cho, S.-J. (2011). An empirical model of mainframe computer investment. *Journal of Applied Econometrics*, 26(1):122–150.
- Cui, K. Z., Demirer, M., Jaffe, S., Muhn, M., Schreiber, M., Soll, J., Stokel-Walker, C., Yeomans, M., Zhang, L., and Brynjolfsson, E. (2026). The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers. *Management Science*.

- Cunningham, W. (1992). The WyCash portfolio management system. *OOPSLA '92: Addendum to the Proceedings*.
- European Parliament and Council of the European Union (2024). Regulation (EU) 2024/1689 of the european parliament and of the council (eu AI act). Official Journal of the European Union.
- Ghaleb, T. A. (2026). Fingerprinting AI coding agents on GitHub. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR 2026)*.
- Han, X. and Xu, L. (2019). Technology adoption in input-output networks. Technical Report Staff Working Paper 2019-51, Bank of Canada.
- Hotz, V. J. and Miller, R. A. (1993). Conditional choice probabilities and the estimation of dynamic models. *Review of Economic Studies*, 60(3):497–529.
- Hotz, V. J., Miller, R. A., Sanders, S., and Smith, J. (1994). A simulation estimator for dynamic models of discrete choice. *Review of Economic Studies*, 61(2):265–289.
- Keane, M. P. and Wolpin, K. I. (1994). The solution and estimation of discrete choice dynamic programming models by simulation and interpolation: Monte Carlo evidence. *Review of Economic Studies*, 61(4):265–289.
- Li, Z., Avgeriou, P., and Liang, P. (2015). A systematic mapping study on technical debt and its management. *Journal of Systems and Software*, 101:193–220.
- Lientz, B. P. and Swanson, E. B. (1980). Software maintenance management. *Communications of the ACM*, 23(5):279–285.
- Liu, Y. et al. (2026). On the code quality of AI-generated code. arXiv preprint arXiv:2603.28592.
- Magnac, T. and Thesmar, D. (2002). Identifying dynamic discrete decision processes. *Econometrica*, 70(2):801–816.
- McElheran, K., Brynjolfsson, E., Yang, L., and Kroff, Z. (2025). Industrial AI and the J-curve of productivity. Wharton Mack Institute Working Paper.
- Microsoft Corporation (2025). Fiscal year 2025 third quarter earnings release. Microsoft Investor Relations.

- Nagappan, N. and Ball, T. (2005). Use of relative code churn measures to predict system defect density. *Proceedings of the 27th International Conference on Software Engineering (ICSE)*.
- Nguyen, H. (2025). Neural network estimation for structural dynamic discrete choice models. Job Market Paper, Georgetown University.
- Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., and Karri, R. (2022). Asleep at the keyboard? assessing the security of GitHub Copilot’s code contributions. In *Proceedings of the 43rd IEEE Symposium on Security and Privacy*.
- Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590*.
- Perry, N., Srivastava, M., Kumar, D., and Boneh, D. (2023). Do users write more insecure code with AI assistants? In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*.
- Ramasubbu, N. and Kemerer, C. F. (2016). Technical debt and the reliability of enterprise software systems: A large-scale empirical study. *Management Science*, 62(5):1487–1510.
- Rust, J. (1987). Optimal replacement of GMC bus engines: An empirical model of harold zurcher. *Econometrica*, 55(5):999–1033.
- Rust, J. (1994). Structural estimation of Markov decision processes. In *Handbook of Econometrics*, volume 4. Elsevier.
- Ryan, S. P. and Tucker, C. (2012). Heterogeneity and the dynamics of technology adoption. *Quantitative Marketing and Economics*, 10(1):63–109.
- Selvanayagam, N., Ghaleb, T. A., and Abdellatif, A. (2026). Self-admitted technical debt in LLM software: An empirical comparison with ML and non-ML software. In *Proceedings of SANER 2026*.
- Zhu, H. et al. (2026). Volume-quality inverse law in AI-generated code. *arXiv preprint arXiv:2605.02741*.